

Experiment No.	1
Experiment Title	Working with Python packages - Numpy, Scipy, Scikit-Learn, Matplotlib ¹
Student Name	Sanjay S
Register Number	3122247001056
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	03/08/2026

1 Objective

To get comfortable with Numpy, Pandas, Scipy, Scikit-learn and Matplotlib, and to practice the standard ML workflow (load, explore, clean, split, evaluate) on five different datasets so the right category of problem can be identified before any modeling is attempted.

2 Problem Statement

Given five public datasets covering different learning problems, load each one, decide whether it needs classification or regression, clean it, split it, and train a small baseline model to confirm the pipeline works end to end.

3 Dataset Description

Dataset	Source	Samples	Features	Classes	Missing
Iris	scikit-learn built in	150	4	3	none
Loan Amount Prediction	Kaggle, philein-sophos/predict-loan-amount-data	29660	20	regression target	24327 cells across several columns
Predicting Diabetes	Kaggle, uciml/pima-indians-diabetes-database	768	8	2	none recorded, though several medical fields use 0 as a placeholder
Email Spam (Spambase)	Kaggle, somesh24/spambase	4601	57	2	none

¹GitHub Repository: <https://github.com/sanjay-6727/Introduction-to-Machine-Learning->

Handwritten Digits	scikit-learn built in (digits, used as an MNIST style stand in)	1797	64	10	none
--------------------	---	------	----	----	------

Each dataset was split roughly 60/20/20 into train, validation and test, stratified where the task was classification.

4 Exploratory Data Analysis

- Dataset overview and statistical summary: histogram, box plot and violin panels per dataset.
- Missing value analysis: loan data has real gaps, the other four are complete.
- Class distribution and correlation: shown via pair plots and correlation heatmaps per dataset.

Experiment_1 - eda (47 images)

Inference:

Iris shows petal length and petal width almost perfectly correlated (about 0.96). The loan missing value heatmap points to Type of Employment and Property Age as the worst gaps. Spambase's word frequency histograms are heavily right skewed with a spike at zero, most emails simply do not use most tracked words. The digit samples confirm the images really are just 8x8 pixels. Box plots show Diabetes and Loan Amount carry more outliers than the cleaner Iris dataset.

5 Data Preprocessing

- Missing values: loan's numeric columns filled with the median, categorical columns with the mode.
- Encoding: loan's categorical columns were label encoded, the other datasets are already numeric.
- Scaling: every feature matrix was standardized with StandardScaler before modeling.
- Split: 60/20/20 train/validation/test, stratified for classification tasks.
- Loan's identifier columns (Customer ID, Name, Property ID) were dropped before modeling.

Listing 1: Same pattern reused across all five datasets: split, scale, fit, evaluate

```

1 X_train, X_test, y_train, y_test = train_test_split(
2     X, y, test_size=0.2, random_state=42, stratify=y)
3
4 scaler = StandardScaler().fit(X_train)
5 X_train_s = scaler.transform(X_train)
6 X_test_s = scaler.transform(X_test)
7
8 model = LogisticRegression(max_iter=1000).fit(X_train_s, y_train)
9 pred = model.predict(scaler.transform(X_test))
10 print(accuracy_score(y_test, pred))

```

6 Machine Learning Algorithm

Four baseline algorithms were used, one per task, just to prove the pipeline works.

Logistic Regression (Iris, Spambase): models $P(y = 1 \mid x) = \frac{1}{1 + e^{-(w^T x + b)}}$, fast and interpretable but limited to roughly linear boundaries.

Linear Regression (Loan Amount): fits $y = w^T x + b$ by minimizing squared error, simple but sensitive to outliers and unscaled features.

Random Forest Classifier (Diabetes): votes across many decision trees, handles non-linear patterns well, slower and harder to explain than a linear model.

K-Nearest Neighbors (Digits): classifies by majority vote among the k closest points, adapts to complex boundaries but gets slower as the training set grows.

7 Experimental Setup

Python Version	3.11.9
Scikit-Learn Version	1.7.2
IDE	Anaconda Navigator / VS Code
Operating System	Windows 11
Hardware	

8 Hyperparameter Selection

Kept at sensible defaults since this experiment is about the workflow, not tuning (that is covered in Experiments 2 to 4).

Dataset	Model	Setting
Iris	Logistic Regression	max_iter = 1000, default L2 penalty
Loan Amount	Linear Regression	no hyperparameters
Diabetes	Random Forest	n_estimators = 200
Spambase	Logistic Regression	max_iter = 2000, default L2 penalty
Digits	K-Nearest Neighbors	k = 5

9 Results

Dataset	Suitable Algorithm(s)	Test Result
Iris	Logistic Regression / KNN / Decision Tree	Accuracy = 0.9333
Loan Amount	Linear/Ridge Regression / Random Forest	R2 = 0.5515
Diabetes	Logistic Regression / Random Forest	Accuracy = 0.7403
Email Spam	Naive Bayes / SVM / Logistic Regression	Accuracy = 0.9229
Digits (MNIST style)	KNN / SVM / CNN	Accuracy = 0.9639

10 Result Visualization

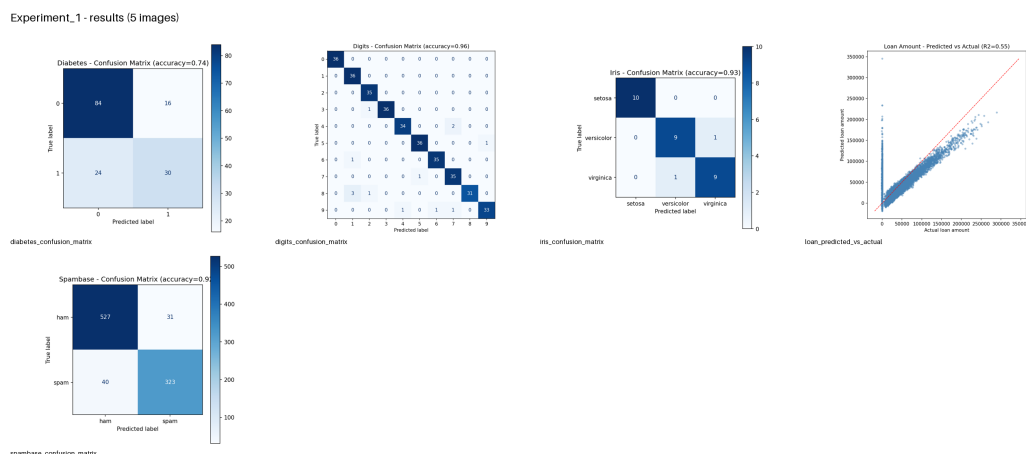


Figure 2: Confusion matrices for Iris, Diabetes, Spambase and Digits, and the predicted vs actual scatter for the Loan Amount regression.

Inference:

Iris's confusion matrix is almost perfectly diagonal, with only a couple of versicolor/virginica mixups, the two species known to overlap slightly. The loan scatter sits close to the diagonal for small to mid loan amounts but drifts wider for the largest ones, fewer big loans means less to learn from. Diabetes shows more false negatives than false positives, worth flagging since missed positives matter more in a medical setting. Spambase and Digits both show strong, mostly clean diagonals.

11 Discussion

The workflow stayed the same across datasets, but the specific technique inside each step changed. None of the five baselines were tuned, so these numbers are a floor, not a ceiling. Diabetes is the weakest result, likely a mix of a small sample size (768 rows) and known data quality issues where zero stands in for missing in fields like insulin and skin thickness.

12 Conclusion

The same six step workflow applied cleanly across three task types, classification, regression and a multi class image problem. Iris and Digits gave the strongest baselines since they are clean and well structured, Loan Amount and Diabetes were harder due to missing data and smaller, noisier features.

13 Learning Outcomes

- Got hands on practice with the core numpy, pandas, scipy, scikit-learn and matplotlib functions used in almost every ML project.

- Saw that preprocessing needs change a lot by dataset, clean data like Iris barely needs anything, messy data like Loan Amount needs real imputation and encoding work.
- Learned to read a confusion matrix for what it actually says about a model, not just the accuracy number.
- Realized that a baseline result without any tuning is still useful, it tells you the floor you are trying to improve on.

14 References

1. Stephen Marsland, “Machine Learning - An Algorithmic Perspective”, 2nd Edition, Chapman and Hall/CRC Machine Learning and Pattern Recognition Series, 2015.
2. Ethem Alpaydin, “Introduction to Machine Learning”, 3rd Edition, The MIT Press, 2014.